

Rijndael Encryption

基于 Ptolemy II 实现

团队成员：

目录

1. AES 标准介绍.....	3
1.1. AES 标准.....	3
1.1.1. AES 标准简介.....	3
1.1.2. AES 标准和 Rijndael 算法的不同.....	3
2. Rijndael 加密算法理论介绍.....	3
2.1. Rijndael 算法的历史及特点.....	3
2.1.1 对称加密算法与非对称加密算法的区别.....	3
2.2. AES 标准下的 Rijndael 算法.....	4
2.3. Rijndael 算法宏观加解密步骤.....	4
2.4. Rijndael 详细加密步骤.....	5
2.4.1. 综述.....	5
2.4.2. Rijndael 整体加解密流程.....	6
2.4.3. Rijndael 加密函数操作阶段.....	8
2.4.4. 密钥扩展算法.....	11
2.5. Rijndael 设计思想.....	12
3. Rijndael 算法在 Ptolemy II 环境下的实现原理.....	12
3.1. 抽象类声明.....	12
3.1.1. convertToStringClass 抽象类.....	12
3.1.2. ShiftRow 抽象类.....	13
3.1.3. MixColumn 抽象类.....	13
3.1.4. MulTwo 抽象类.....	14
3.1.5. MulThree 抽象类.....	14
3.1.6. rotate bytes 抽象类.....	15
3.2. 算法层次说明.....	15
3.2.1. 应用层.....	15
3.2.2. Rijndael 层.....	16
3.2.3. 密钥扩展算法层 (RoundKeyGenerator).....	17
3.2.4. 轮函数层 (RoundSequence).....	18
4. Rijndael 算法在 Ptolemy II 环境下的实验结果.....	19
4.1. 实验模型.....	19
4.1.1. 抽象层.....	19
4.1.2. 应用层.....	19
4.1.3. Rijndael 层.....	19
4.1.4. 密钥扩展算法层.....	19
4.1.5. 轮函数层.....	19
4.2. 实验模型输入输出.....	19
4.2.1. 初始输入.....	19
4.2.2. 中间结果输出.....	20
4.2.3. 最终结果输出.....	20
5. 团队成员及任务分工.....	20
6. 实验收获.....	20
7. 参考文献.....	20

1. AES 标准介绍

1.1. AES 标准

1.1.1. AES 标准简介

AES, Advanced Encryption Standard, 其实是一套标准: FIPS 197, 而我们所说的 AES 算法其实是 Rijndael 算法。

NIST(National INstitute of Standards and Technology)在 1997 年 9 月 12 日公开征集更高效更安全的替代 DES 加密算法, 第一轮共有 15 种算法入选, 其中 5 种算法入围了决赛, 分别是 MARS, RC6, Rijndael, Serpent 和 Twofish。又经过 3 年的验证、评测及公众讨论之后 Rijndael 算法最终入选。

1.1.2. AES 标准和 Rijndael 算法的不同

Rijndael 算法支持多种分组及密钥长度, 介于 128-256 之间所有 32 的倍数均可, 最小支持 128 位, 最大 256 位, 共 25 种组合。而 AES 标准支持的分组大小固定为 128 位, 密钥长度有 3 种选择: 128 位、192 位及 256 位。为方便起见, 以下仅介绍 AES 标准下的 Rijndael 算法, 且规定 AES 与 Rijndael 的含义相同。

2. Rijndael 加密算法理论介绍

2.1. Rijndael 算法的历史及特点

Rijndael 算法是由比利时学者 Joan Daemen 和 Vincent Rijmen 所提出的, 算法的名字就由两位作者的名字组合而成。Rijndael 的优势在于集安全性、性能、效率、可实现性及灵活性于一体。Rijndael 算法是**对称加密算法**。对称加密算法就是加密和解密使用相同的密钥。

2.1.1 对称加密算法与非对称加密算法的区别

对称加密算法的加密和解密用到的密钥是相同的, 这种加密方式加密速度非常快, 适合经常发送数据的场合。缺点是密钥的传输比较麻烦。

非对称加密算法的加密和解密用的密钥是不同的, 这种加密方式是用数学上的难解问题构造的, 通常加密解密的速度比较慢, 适合偶尔发送数据的场合。优点是密钥传输方便。常见的非对称加密算法为 RSA、ECC 和 ElGamal。

实际中, 一般是通过 RSA 加密 AES 的密钥, 传输到接收方, 接收方解密得到 AES 密钥, 然后发送方和接收方用 AES 密钥来通信。

2.2. AES 标准下的 Rijndael 算法

Rijndael 为分组密码，分组密码也就是把明文分成一组一组的，每组长度相等，每次加密一组数据，直到加密完整个明文。在 AES 标准规范中，分组长度只能是 128 位，也就是说，每个分组为 16 个字节（每个字节 8 位）。密钥的长度可以使用 128 位、192 位或 256 位。密钥的长度不同，推荐加密轮数也不同，如下表所示：

AES	密钥长度(32bits)	分组长度(32bits)	加密轮数
AES-128	4	4	10
AES-192	6	4	12
AES-256	8	4	14

表 2.2-1 Rijndael 算法的 AES 标准

考虑到实现难度和实现时间的问题以及本课的重点，本次实验选用 AES-128 标准以说明问题。在下面的介绍中，也只介绍 AES-128 标准，并且只介绍算法的加密部分。

2.3. Rijndael 算法宏观加解密步骤

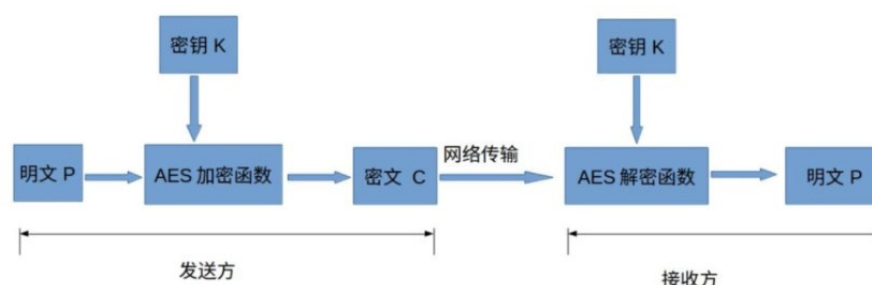


图 2.3-1 Rijndael 算法宏观加解密步骤

上图各部分的作用与意义：

1. 明文 P：

没有经过加密的数据。

2. 密钥 K：

用来加密明文的密码，在对称加密算法中，加密与解密的密钥是相同的。密钥为接收方与发送方协商产生，但不可以直接在网络上传输，否则会导致密钥泄漏，通常是通过非对称加密算法加密密钥，然后再通过网络传输给对方，或者直接面对面商量密钥。密钥是绝对不可以泄漏的，否则会被攻击者还原密文，窃取机密数据。

3. AES 加密函数：

设 AES 加密函数为 E，则 $C=E(K,P)$ ，其中 P 为明文，K 为密钥，C 为密文。也就是说，把明文 P 和密钥 K 作为加密函数的参数输入，则加密函数 E 会输出密文 C。

4. 密文 C：

经加密函数处理后的数据。

5. AES 解密函数：

设 AES 解密函数为 D，则 $P=D(K,C)$ ，其中 C 为密文，K 为密钥，P 为明文。也就是说，把密文 C 和密钥 K 作为解密函数的参数输入，则解密函数会输出明文 P。

2.4. Rijndael 详细加密步骤

2.4.1. 综述

上面说到，Rijndael 的加密公式为 $C=E(K,P)$ ，在加密函数 E 中，会先进行一次异或加密操作，然后把一个轮函数执行 10 次，这个轮函数的前 9 次执行的操作是一样的，只有第 10 次有所不同。也就是说，一个明文分组会被加密 10 轮。Rijndael 的核心就是实现一轮中的所有操作。

Rijndael 的处理单位是字节，128 位的输入明文分组 P 和输入密钥 K 都被分成 16 个字节，分别记为 $P=P_0 P_1 \cdots P_{15}$ 和 $K=K_0 K_1 \cdots K_{15}$ 。如，明文分组为 $P = \text{abcdefghijklmnop}$ ，其中的字符 a 对应 P_0 ， p 对应 P_{15} 。一般地，明文分组用字节为单位的正方形矩阵描述，称为**状态矩阵**。在算法的每一轮中，状态矩阵的内容不断发生变化，最后的结果作为密文输出。该矩阵中字节的排列顺序为**从上到下、从左至右**依次排列，如下图所示：

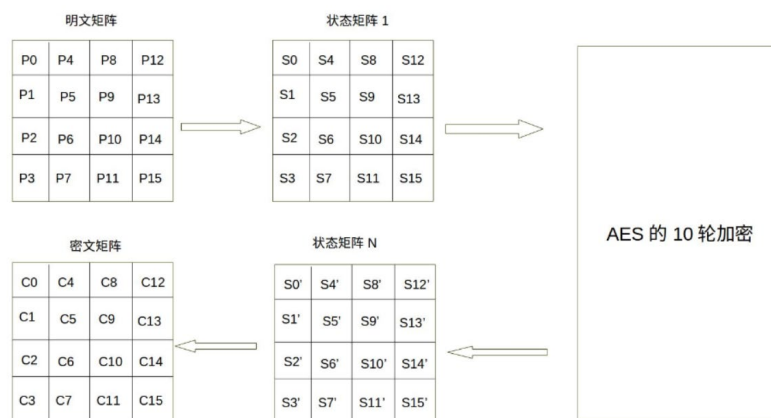


图 2.4.1-1 状态矩阵示意图

现在假设明文分组 P 为 "abcdefghijklmnop"，则对应上面生成的状态矩阵图如下：

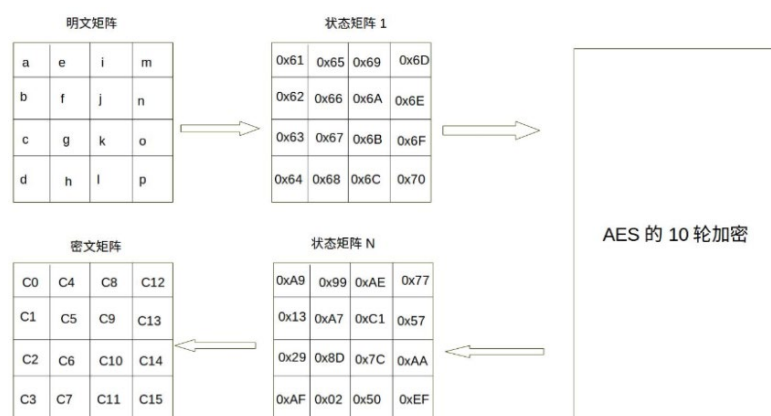


图 2.4.1-2 状态矩阵实例图

上图中，0x61 为字符 a 的十六进制表示。可以看到，明文经过 AES 加密后，已经面目全非。

类似地，128 位密钥也是用字节为单位的矩阵表示，矩阵的每一列被称为 1 个 32 位比特字。通过密钥编排函数该密钥矩阵被扩展成一个 44 个字组成的序列 $W[0], W[1], \cdots, W[43]$ ，该序列的前 4 个元素 $W[0], W[1], W[2], W[3]$ 是原始密钥，用于加密

运算中的初始密钥加（下面介绍）；后面 40 个字分为 10 组，每组 4 个字（128 比特）分别用于 10 轮加密运算中的轮密钥加，如下图所示：

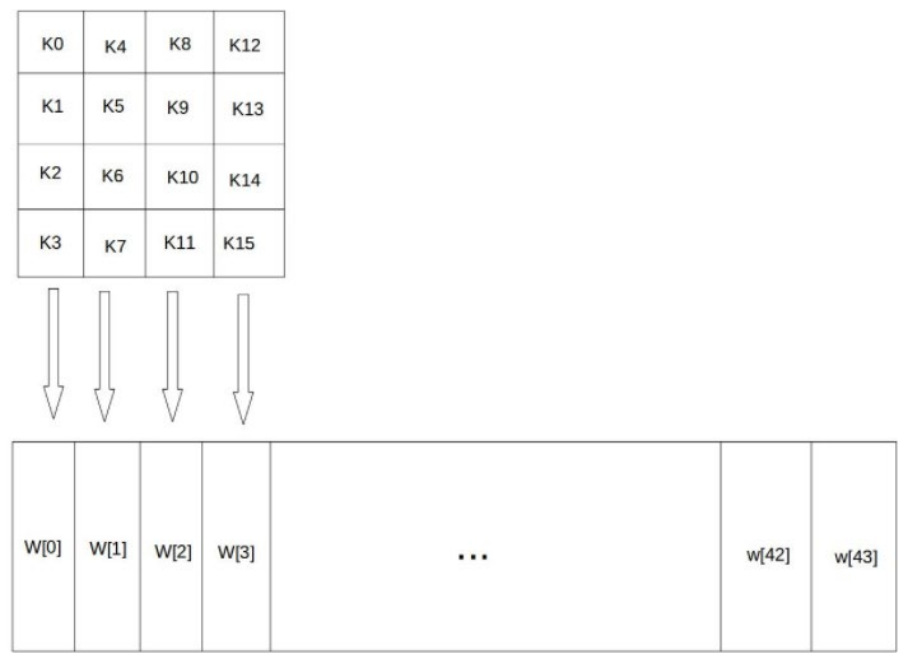


图 2.4.1-3 密钥矩阵扩展序列图

上图中，设 K = “abcdefghijklmnop”，则 K0 = a，K15 = p，W[0] = K0 K1 K2 K3 = “abcd”。

2.4.2. Rijndael 整体加解密流程

Rijndael 的加解密整体流程如下图所示，其中的 W[0,3]是指 W[0]、W[1]、W[2]和 W[3]串联组成的 128 位密钥。加密的第 1 轮到第 9 轮的轮函数一样，包括 4 个操作：**字节代换、行位移、列混合和轮密钥加**。最后一轮迭代不执行列混合。另外，在第一轮迭代之前，先将明文和原始密钥进行一次**异或加密操作**。

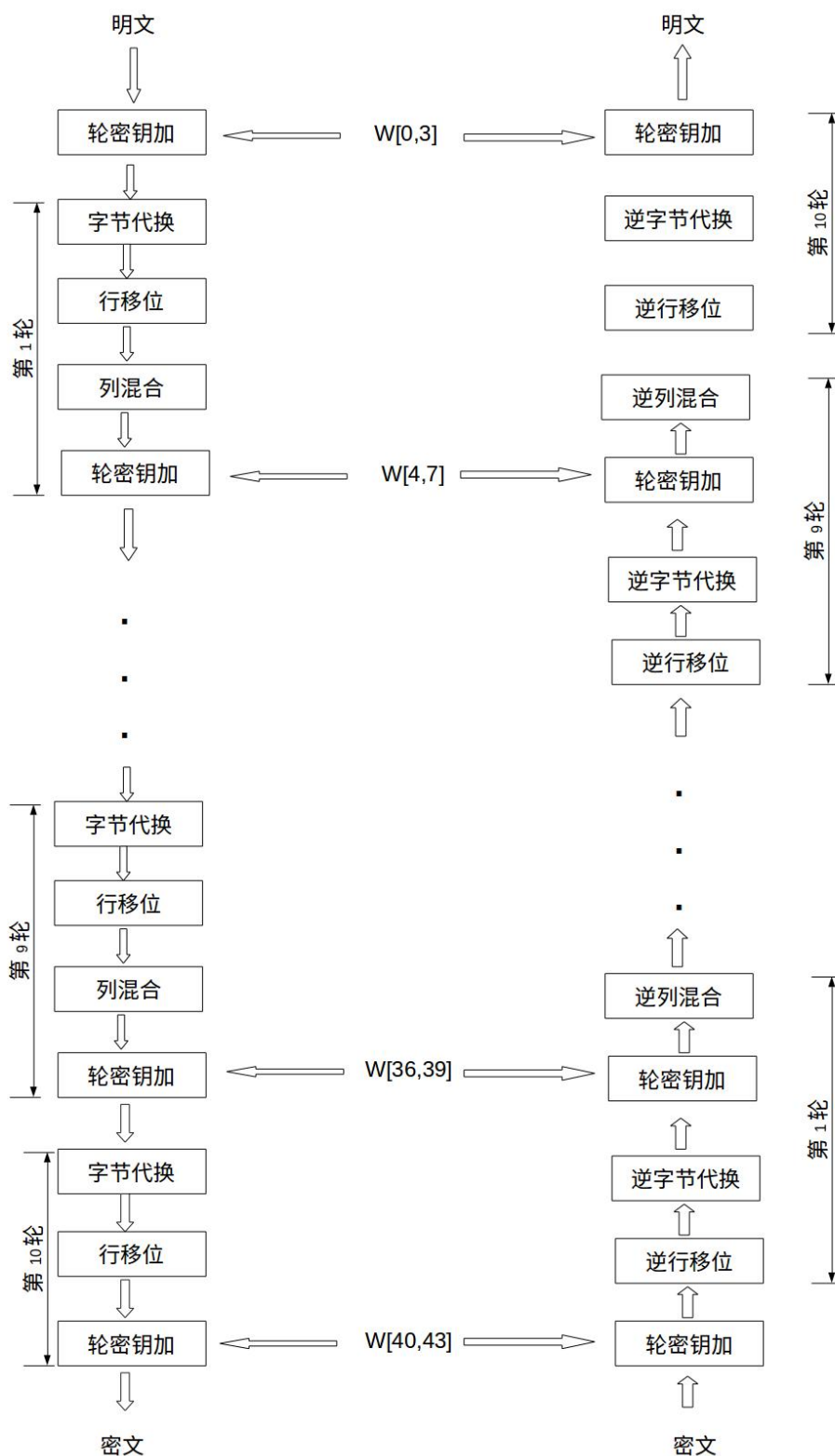


图 2.4.2-1 状态矩阵实例图

上图也展示了 Rijndael 解密过程，解密过程仍为 10 轮，每一轮的操作是加密操作的逆操作。由于 Rijndael 的 4 个轮操作都是可逆的，因此，解密操作的一轮就是顺序执行逆行移位、逆字节代换、轮密钥加和逆列混合。同加密操作类似，最后一轮不执行逆列混合，在第 1 轮解密之前，要执行 1 次密钥加操作。当然这部分内容，本文不涉及。

下面分别介绍 Rijndael 函数中一轮的 4 个操作阶段，这 4 分操作阶段使输入位得到充

分的混淆。

2.4.3. Rijndael 加密函数操作阶段

2.4.3.1. 字节代换操作

字节代换 (SubBytes) 是对 state 矩阵中的每一个独立元素于置换盒 (Substitution-box, S 盒) 中进行查找并以此替换输入状态的操作。字节代换是可逆的非线性变换, 也是 AES 运算组中唯一的非线性变换。字节代换逆操作也是通过逆向置换盒的查找及替换来完成的。

S 盒是事先设计好的 16x16 的查询表, 即 256 个元素。其设计不是随意的, 要根据设计原则严格计算求得, 不然无法保证算法的安全性。既然是 S 盒是计算得来, 所以字节代换的操作完全可以通过计算来完成, 不过通过 S 盒查表操作更方便快捷。

Rijndael 的 S 盒:

列号 行号	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0x63	0x7C	0x77	0x7B	0xF2	0x6B	0x6F	0xC5	0x30	0x01	0x67	0x2B	0xFE	0xD7	0xAB	0x76
1	0xCA	0x82	0xC9	0x7D	0xFA	0x59	0x47	0xF0	0xAD	0xD4	0xA2	0xAF	0x9C	0xA4	0x72	0xC0
2	0xB7	0xFD	0x93	0x26	0x36	0x3F	0xF7	0xCC	0x34	0xA5	0xE5	0xF1	0x71	0xD8	0x31	0x15
3	0x04	0xC7	0x23	0xC3	0x18	0x96	0x05	0x9A	0x07	0x12	0x80	0xE2	0xEB	0x27	0xB2	0x75
4	0x09	0x83	0x2C	0x1A	0x1B	0x6E	0x5A	0xA0	0x52	0x3B	0xD6	0xB3	0x29	0xE3	0x2F	0x84
5	0x53	0xD1	0x00	0xED	0x20	0xFC	0xB1	0x5B	0x6A	0xCB	0xBE	0x39	0x4A	0x4C	0x58	0xCF
6	0xD0	0xEF	0xAA	0xFB	0x43	0x4D	0x33	0x85	0x45	0xF9	0x02	0x7F	0x50	0x3C	0x9F	0xA8
7	0x51	0xA3	0x40	0x8F	0x92	0x9D	0x38	0xF5	0xBC	0xB6	0xDA	0x21	0x10	0xFF	0xF3	0xD2
8	0xCD	0x0C	0x13	0xEC	0x5F	0x97	0x44	0x17	0xC4	0xA7	0x7E	0x3D	0x64	0x5D	0x19	0x73
9	0x60	0x81	0x4F	0xDC	0x22	0x2A	0x90	0x88	0x46	0xEE	0xB8	0x14	0xDE	0x5E	0x0B	0xDB
10	0xE0	0x32	0x3A	0x0A	0x49	0x06	0x24	0x5C	0xC2	0xD3	0xAC	0x62	0x91	0x95	0xE4	0x79
11	0xE7	0xC8	0x37	0x6D	0x8D	0xD5	0x4E	0xA9	0x6C	0x56	0xF4	0xEA	0x65	0x7A	0xAE	0x08
12	0xBA	0x78	0x25	0x2E	0x1C	0xA6	0xB4	0xC6	0xE8	0xDD	0x74	0x1F	0x4B	0xBD	0x8B	0x8A
13	0x70	0x3E	0xB5	0x66	0x48	0x03	0xF6	0x0E	0x61	0x35	0x57	0xB9	0x86	0xC1	0x1D	0x9E
14	0xE1	0xF8	0x98	0x11	0x69	0xD9	0x8E	0x94	0x9B	0x1E	0x87	0xE9	0xCE	0x55	0x28	0xDF
15	0x8C	0xA1	0x89	0x0D	0xBF	0xE6	0x42	0x68	0x41	0x99	0x2D	0x0F	0xB0	0x54	0xBB	0x16

图 2.4.3.1-1 S 盒

状态矩阵中的元素按照下面的方式映射为一个新的字节: 把该字节的高 4 位作为行值, 低 4 位作为列值, 取出 S 盒中对应的行的元素作为输出。例如, 加密时, 输出的字节 S1 为 0x12, 则查 S 盒的第 0x01 行和 0x02 列, 得到值 0xc9, 然后替换 S1 原有的 0x12 为 0xc9。状态矩阵经字节代换后的图如下:

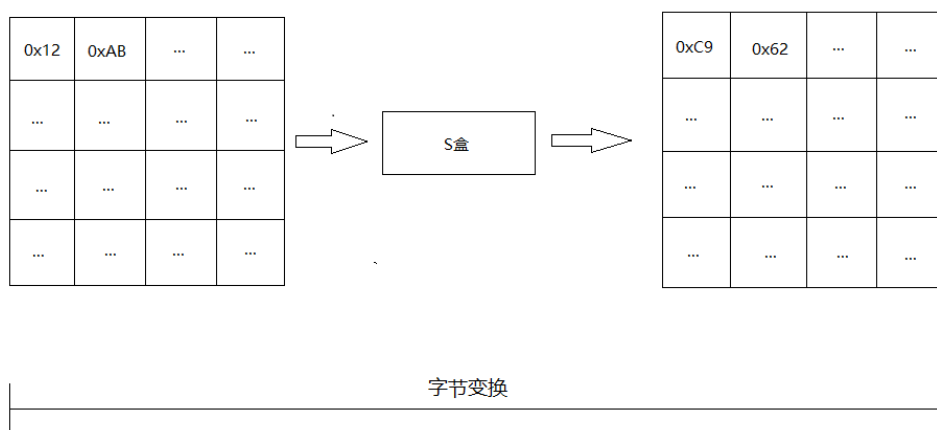


图 2.4.3.1-2 状态矩阵字节代换示意图

2.4.3.2. 行移位操作

行移位主要目的是实现字节在每一行的扩散，属于线性变换。当密钥长度为 128 比特时，状态矩阵的第 0 行左移 0 字节，第 1 行左移 1 字节，第 2 行左移 2 字节，第 3 行左移 3 字节，如下图所示：

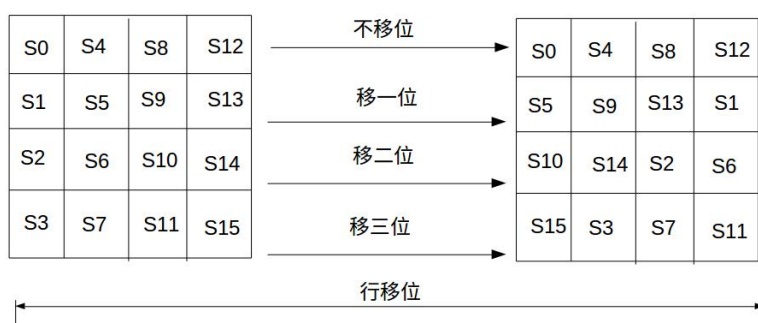


图 2.4.3.2-1 行移位示意图

2.4.3.3. 列混合操作

列混合是通过将经移位后的状态矩阵矩阵与常矩阵 C 相乘以达成在列上的扩散，属于代替变换。列混合是 Rijndael 算法中最复杂的一步，其实质是在有限域 GF(256) 上的多项式乘法运算。具体公式如下图：

$$\begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}^{\omega}$$

图 2.4.3.3-1 列混合矩阵公式

状态矩阵中的第 j 列 ($0 \leq j \leq 3$) 的列混合可以表示为下图所示：

$$\begin{aligned}
s'_{0,j} &= (2 * s_{0,j}) \oplus (3 * s_{1,j}) \oplus s_{2,j} \oplus s_{3,j} \\
s'_{1,j} &= s_{0,j} \oplus (2 * s_{1,j}) \oplus (3 * s_{2,j}) \oplus s_{3,j} \\
s'_{2,j} &= s_{0,j} \oplus s_{1,j} \oplus (2 * s_{2,j}) \oplus (3 * s_{3,j}) \\
s'_{3,j} &= (3 * s_{0,j}) \oplus s_{1,j} \oplus s_{2,j} \oplus (2 * s_{3,j})
\end{aligned}$$

图 2.4.3.3-2 列混合分解公式

其中，矩阵元素的乘法和加法都是定义在基于 $GF(2^8)$ 上的二元运算，并不是通常意义上的乘法和加法。这种二元运算的加法等价于两个字节的异或，乘法则复杂一点。对于一个 8 位的二进制数来说，使用域上的乘法乘以 (00000010) 等价于左移 1 位 (低位补 0) 后，再根据情况同 (00011011) 进行异或运算，设 $S1 = (a_7 \ a_6 \ a_5 \ a_4 \ a_3 \ a_2 \ a_1 \ a_0)$ ，则 $0x02 * S1$ 如下图所示：

$$(00000010) * (a_7 a_6 a_5 a_4 a_3 a_2 a_1 a_0) = \begin{cases} (a_6 a_5 a_4 a_3 a_2 a_1 a_0 0), & a_7 = 0 \\ (a_6 a_5 a_4 a_3 a_2 a_1 a_0 0) \oplus (00011011), & a_7 = 1 \end{cases}$$

图 2.4.3.3-3 矩阵元素乘法定义

2.4.3.4. 轮密钥加操作

轮密钥加是将 128 位轮密钥 K_i 同状态矩阵中的数据进行逐位异或操作，如下图所示。其中，密钥 K_i 中每个字 $W[4i], W[4i+1], W[4i+2], W[4i+3]$ 为 32 位比特字，包含 4 个字节，他们的生成算法下面在下面介绍。轮密钥加过程可以看成是字逐位异或的结果，也可以看成字节级别或者位级别的操作。也就是说，可以看成 $S0 \ S1 \ S2 \ S3$ 组成的 32 位字与 $W[4i]$ 的异或运算。

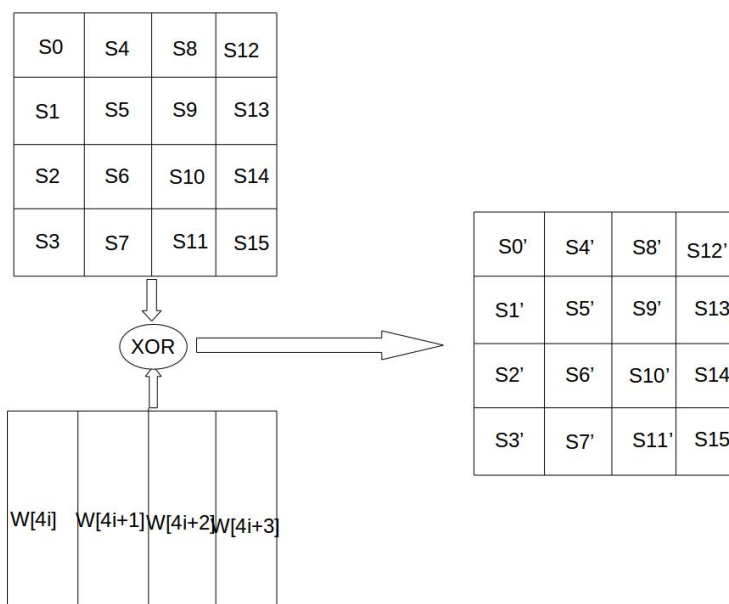


图 2.4.3.4-1 轮密钥加操作示意图

轮密钥加的逆运算同正向的轮密钥加运算完全一致，这是因为异或的逆操作是其自身。轮密钥加非常简单，但却能够影响 S 数组中的每一位。

2.4.4. 密钥扩展算法

AES 首先将初始密钥输入到一个 44 的状态矩阵中，如下图所示：

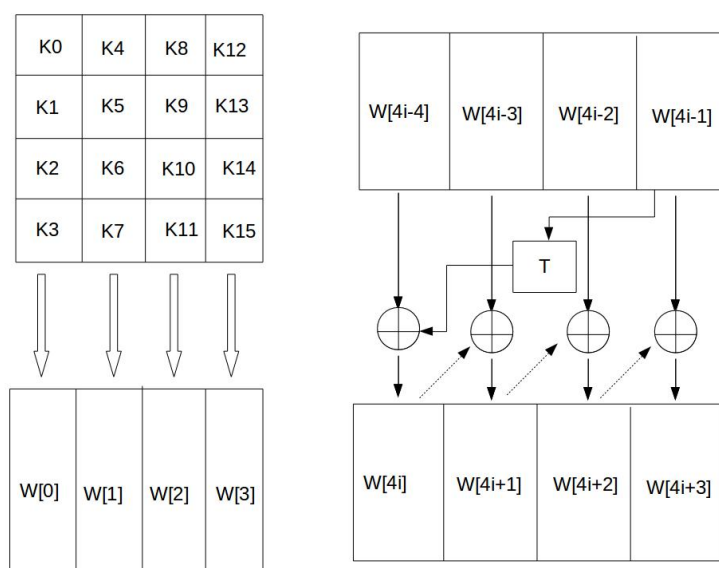


图 2.4.4-1 密钥扩展算法示意图

这个 4*4 矩阵的每一列的 4 个字节组成一个字，矩阵 4 列的 4 个字依次命名为 W[0]、W[1]、W[2]和 W[3]，它们构成一个以字为单位的数组 W。

例如，设密钥 K 为 "abcdefghij klmnop", 则 K0= 'a' , K1= 'b' , K2= 'c' , K3= 'd' , W[0]= "abcd" 。

接着，对 W 数组扩充 40 个新列，构成总共 44 列的扩展密钥数组。新列以如下的递归方式产生：

1. 如果 i 不是 4 的倍数，那么第 i 列由如下等式确定：

$$W[i] = W[i-4] \oplus W[i-1]$$

2. 如果 i 是 4 的倍数，那么第 i 列由如下等式确定：

$$W[i] = W[i-4] \oplus T(W[i-1])$$

其中，T 是一个有点复杂的函数。

函数 T 由 3 部分组成：字循环、字节代换和轮常量异或，这 3 部分的作用分别如下。

a. 字循环：将 1 个字中的 4 个字节循环左移 1 个字节。即将输入字 [b0, b1, b2, b3] 变换成 [b1, b2, b3, b0]。

b. 字节代换：对字循环的结果使用 S 盒进行字节代换。

c. 轮常量异或：将前两步的结果同轮常量 Rcon[j] 进行异或，其中 j 表示轮数。轮常量 Rcon[j] 是一个字，其值见下表。

j	1	2	3	4	5
Rcon[j]	01 00 00 00	02 00 00 00	04 00 00 00	08 00 00 00	10 00 00 00
j	6	7	8	9	10
Rcon[j]	20 00 00 00	40 00 00 00	80 00 00 00	1B 00 00 00	36 00 00 00

表 2.4.4-1 轮常量

下面一例说明问题：

设初始的 128 位密钥为：3C A1 0B 21 57 F0 19 16 90 2E 13 80 AC C1 07 BD，那么 4 个初始值为：W[0] = 3C。

2.5. Rijndael 设计思想

1. 安全性 (Security) 算法足够强，抗攻击。
2. 经济性 (Efficiency) 算法运算效率高。
3. 密钥捷变 (Key Agility) 更改密钥所引入的损失尽量小，即最小消耗的密钥扩展算法。
4. 适应性 (Versatility) 适用于不同的 CPU 架构，软件或硬件平台的实现。
5. 设计简单 (Simplicity) 轮函数的设计精简，只是多轮迭代。

3. Rijndael 算法在 Ptolemy II 环境下的实现原理

3.1. 抽象类声明

3.1.1. convertToStringClass 抽象类

3.1.1.1. 输入输出

输入：一个字节

输出：32 个十六进制字符

3.1.1.2. 功能

将字节序列转换为字符串，一次一个字节，每个字节填充为两个十六进制数字

3.1.1.3. 功能实现

将输入的一个字节，通过 Expression 和 PadToTwoChars 转化为一个两位的十六进制字符串，在 Accumulator 处累加，DownSample 实现取每 16 个这样的字符最后一个字符串；Const、SampleDelay、UpSample 实现每取样 16 次，进行一次 Accumulator 重置，

SampleDelay 初始化输出为 true，UpSample 实现每十六次进行一次对 SampleDelay 输出结果取样；并且 SDF Director 的 iterations 参数置 0，实现有输入即可响应。

3.1.2. ShiftRow 抽象类

3.1.2.1. 输入输出

输入：16 进制字符串

输出：16 进制字符串

3.1.2.2. 功能

行移位

3.1.2.3. 功能实现

SequenceToArray 实现将输入的每 16 个字符串转换为四个长度为四的数组，Distributor 实现将数组等分为四组；Expression 实现第一个数组不变，第二个数组向左偏移一位，第三个数组向左偏移两位，第四个数组向左偏移三位，具体实现如下图；ArrayToString 实现将数组转化为字符串。

3.1.3. MixColumn 抽象类

3.1.3.1. 输入输出

输入：16 进制字符串

输出：16 进制字符串

3.1.3.2. 功能

列混淆

3.1.3.3. 功能实现

实现公式：

$$\begin{aligned}
s'_{0,j} &= (2 * s_{0,j}) \oplus (3 * s_{1,j}) \oplus s_{2,j} \oplus s_{3,j} \\
s'_{1,j} &= s_{0,j} \oplus (2 * s_{1,j}) \oplus (3 * s_{2,j}) \oplus s_{3,j} \\
s'_{2,j} &= s_{0,j} \oplus s_{1,j} \oplus (2 * s_{2,j}) \oplus (3 * s_{3,j}) \\
s'_{3,j} &= (3 * s_{0,j}) \oplus s_{1,j} \oplus s_{2,j} \oplus (2 * s_{3,j})
\end{aligned}$$

具体实现：

$$\begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}$$

3.1.4. MulTwo 抽象类

3.1.4.1. 输入输出

输入：一个两位的 16 进制数

输出：一个两位的 16 进制数

3.1.4.2. 功能

列混淆中的一个模块

3.1.4.3. 功能实现

当输入的 16 进制数大于 127（十进制）时，将该数向左偏移一位后与 27（十进制）异或，得到输出；若小于等于 127（十进制），将该数向左偏移一位得到输出。

3.1.5. MulThree 抽象类

3.1.5.1. 输入输出

输入：一个两位的 16 进制数

输出：一个两位的 16 进制数

3.1.5.2. 功能

列混淆中的一个模块

3.1.5.3. 功能实现

当输入的 16 进制数大于 127（十进制）时，将该数向左偏移一位后与 27（十进制）异或，得到的结果与他本身异或得到输出；若小于等于 127（十进制），将该数向左偏移一位得到的结果与他本身异或得到输出。

3.1.6. rotate bytes 抽象类

3.1.6.1. 输入输出

输入：32 位的十六进制字符串

输出：32 位的十六进制字符串

3.1.6.2. 功能

循环左移一位

3.1.6.3. 功能实现

将字符串 a b c d 转换为 b c d a

3.2. 算法层次说明

3.2.1. 应用层

3.2.1.1. 概述

实现算法，处理结果并输出。

3.2.1.2. 角色

3.2.1.2.1. Ramp

产生阶跃信号。

3.2.1.2.2. rijndael

算法实现，具体实现将在下文具体阐述。

3.2.1.2.3. converToString

见 3.1.1 节。

3.2.1.2.4. Display

显示结果。

3.2.1.3. 指示器

SDF Director 是同步数据流域，当一个角色执行时，它消耗固定数量的输入数据，并对每一个输出端口产生固定数量的输出数据。其中迭代次数为 2。

3.2.2. Rijdael 层

3.2.2.1. 概述

Rijndael 算法实现，SequenceToArray、UpSample、ArrayToSample3 实现初始变换，RoundSequence 实现 10 轮加密循环；Display 输出中间结果，共 22 行，包含 2 次迭代，每次迭代包含 10 轮循环。

3.2.2.2. 角色

3.2.2.2.1. UpSample、DownSample

进行 10 轮加密，第十一次点火是为了输出。

3.2.2.2.2. RoundSequence

10 轮循环运算，详细见下文。

3.2.2.2.3. RoundKeyGenerator

获得密钥，详细见下文。

3.2.2.2.4. SampleDelay

Init 的 `vectorizationfactor` 为 16，实现每十六次对 RoundSequence 初始化。

3.2.3. 密钥扩展算法层 (RoundKeyGenerator)

3.2.3.1. 概述

获得密钥，通过密钥编排函数该密钥矩阵被扩展成一个字组成的序列，该序列的前四个元素是原始密钥，用于加密运算中的初始密钥加，后面 40 个字分为 10 组，每组 4 个字，分别用于 10 轮加密运算中的轮密钥加。原理如下见 2.4.4 节。

3.2.3.2. 角色

3.2.3.2.1. SampleDelay 、 SampleDelay2

SampleDelay2 输出 $W[4i-1]$ ，SampleDelay 输出 $W[4i-4]$ ， $W[4i-3]$ ， $W[4i-2]$ 。

3.2.3.2.2. modal model

实现 $w[i-1]$ 的变换，见下文。

3.2.3.3. 状态迁移和状态细化子层 (modal model)

3.2.3.3.1. 概述

如果 i 不是 4 的倍数，`donothing`；如果 i 是 4 的倍数，`subrotword`。

3.2.3.3.2. 状态

1. SubRotWord 实现 T 函数，详细见下文。
2. DoNothing 输出不变。

3.2.3.3.3. SubRotWord

实现 T 函数，具体在 SubRotWord。

InstanceOfrotat bytes、InstanceOfSBox 实现字节代换，Expression 实现轮常量异或，InstanceOfMulTwo、SampleDelay、Cons 实现轮常量，功能具体实现见 2.4.4 节。

3.2.4. 轮函数层 (RoundSequence)

3.2.4.1. 概述

实现十轮循环。

3.2.4.2. 状态迁移和细化子层

3.2.4.2.1. init

实现初始变换，input#roundkey。

3.2.4.2.2. regular

实现字节代换、行移位、列混合、轮密钥加。

3.2.4.2.3. final

最终轮，形成最后的密文。

4. Rijndael 算法在 Ptolemy II 环境下的实验结果

4.1. 实验模型

4.1.1. 抽象层

4.1.2. 应用层

4.1.3. Rijndael 层

4.1.4. 密钥扩展算法层

4.1.4.1. 状态迁移子层

4.1.4.2. 状态细化子层

4.1.5. 轮函数层

4.1.5.1. 状态迁移子层

4.1.5.2. 状态细化子层

4.2. 实验模型输入输出

4.2.1. 初始输入

4.2.2. 中间结果输出

4.2.3. 最终结果输出

5. 团队成员及任务分工

6. 实验收获

7. 参考文献

<https://blog.csdn.net/chenggiuming/article/details/82192882>

<https://blog.csdn.net/ayang1986/article/details/83104763>

<https://blog.csdn.net/u010037269/article/details/123863979>

<https://wenku.baidu.com/view/98b0eb035bfb770bf78a6529647d27284a733774.html>

<https://ptolemy.berkeley.edu/>